

- Huffman encoding is optimal.
→ The tree and the corresponding encoding is optimal.

$$\underbrace{ABL_{Hc}}_{T^*} \leq ABL_T \quad \forall T \text{ that follows "not prefix" property.}$$

- Proof by induction on $|S|$:
 $S \leftarrow$ set of letters (alphabet)
 $P \leftarrow$ set of freq.s

BASE CASE: $|S| = 1$
trivial

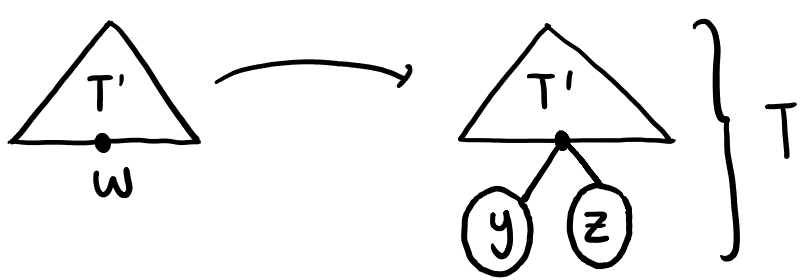
INDUCTION HYPOTHESIS: True $\forall S$ s.t. $|S| \leq k-1$ ↗ no. of elements in S

For the sake of contradiction, Huffman encoding gives a tree T but $\exists Z$ s.t. $ABL_Z < ABL_T$ — *

- Picks two least freq. elements y and z and replaces them in S with a new element ' w '.

$$S' \leftarrow (S / \{y, z\}) \cup \{w\} \qquad |S'| = k-1$$

From I.H., we get an optimal tree T' :



$$\begin{aligned} ABL_T(S) &= \sum_{u \in S} \text{freq}(u) \cdot \text{depth}_T(u) \\ &= \underbrace{\sum_{u \in S \setminus \{y, z\}} \text{freq}(u) \cdot \text{depth}_T(u)}_{S' \setminus \{w\}} + \underbrace{(\text{freq}(y) + \text{freq}(z))}_{\text{freq}(w)} \cdot \underbrace{\text{depth}_T(y)}_{\text{depth}_{T'}(w) + 1} \\ &\quad \forall u \in S \setminus \{y, z\}, \text{depth}_T(u) = \text{depth}_{T'}(u) \\ &\quad \text{depth}_T(y) = \text{depth}_{T'}(w) + 1 \\ &= \sum_{u \in S' \setminus \{w\}} \text{freq}(u) \cdot \text{depth}_{T'}(u) + \text{freq}(w) \cdot (\text{depth}_{T'}(w) + 1) \\ &= \sum_{u \in S'} \text{freq}(u) \cdot \text{depth}_{T'}(u) + \text{freq}(w). \\ &= ABL_{T'}(S') + \text{freq}(w). \quad \text{--- (1)} \end{aligned}$$

Let us consider Z .

- Where do you think ' y ' and ' z ' are in tree Z ? THREE CASES

Without loss of generality, assume that y and z are siblings in the tree Z .
Obtain Z' by replacing parent of ' y ' and ' z ' with w .

$$ABL_Z(S) = ABL_{Z'}(S') + \text{freq}(w) \quad \text{--- (2)}$$

$$\begin{aligned} ABL_{Z'}(S') + \text{freq}(w) &= ABL_Z(S) < ABL_T(S) = ABL_{T'}(S') + \text{freq}(w). \\ &\Downarrow \\ &ABL_{Z'}(S') < ABL_{T'}(S') \\ &\text{This contradicts the optimality of } T'. \end{aligned}$$

• INTERVAL SCHEDULING

Premise: We have one processor and a set of requests/jobs.
· We are given a set of requests as a list of intervals.
· We say a subset of jobs is compatible if they do not have an overlap "pairwise".
Assuming all jobs have the same priority.
Want: Largest no. of compatible jobs.

STRATEGY: earliest finish time (greedy approach)

Algo (R):
 $A \leftarrow \emptyset$
While R is not empty:
· Choose a request i w/ least finish time.
· $A \leftarrow A \cup \{i\}$
· Remove all jobs that are incompatible with i from R along with i .
Return A .

- CORRECTNESS:
The algo. returns the subset $A = \{I_1, I_2, \dots, I_k\}$.
For the sake of contradiction, let us assume
there is a set $O = \{J_1, J_2, \dots, J_m\}$, $m > k$.
We want to prove $m = k$.

$f(I)$ denotes finish time of I .

Our greedy rule guarantees $f(I_1) \leq f(J_1)$

Lemma: $\forall r \leq k, f(I_r) \leq f(J_r)$

$\left\{ \begin{aligned} &\Rightarrow J_{k+1}, \dots, J_m \text{ are compatible w/ } A \\ &\text{This contradicts the termination condition of algo.} \end{aligned} \right.$

Proof by induction on $r \in \{1, \dots, k\}$.
Base case is given by the claim.

I.H.: $f(I_{r-1}) \leq f(J_{r-1})$

I.S.: After the $(r-1)^{\text{th}}$ job finishes in A ,
it picks a job I_r with earliest finish time.

Then, $f(I_r) \leq f(J^*) \quad \forall J^*$ compatible with I_{r-1}
in the updated set R .

In fact, J_r is one such job.

In other words, if $f(J_r) < f(I_r)$,
then algo must choose J_r instead of I_r .

But algo chose $I_r \Rightarrow \underline{\underline{f(I_r) \leq f(J_r)}}$.

Since $m > k$, $\exists J_{k+1}$ in O .

J_{k+1} starts after J_k ends
 $\Rightarrow J_{k+1}$ starts after I_k ends
($f(I_k) \leq f(J_k)$)

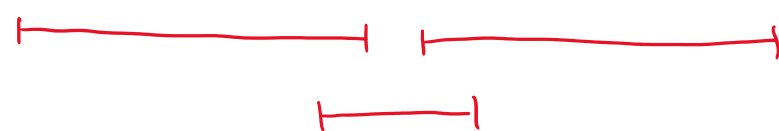
J_{k+1} remains after deleting all requests that are
incompatible with $I_1, \dots, I_k$.
The algorithm would not terminate if such a J_{k+1} existed.
(terminates only when R is empty).

Other greedy solutions which don't work:

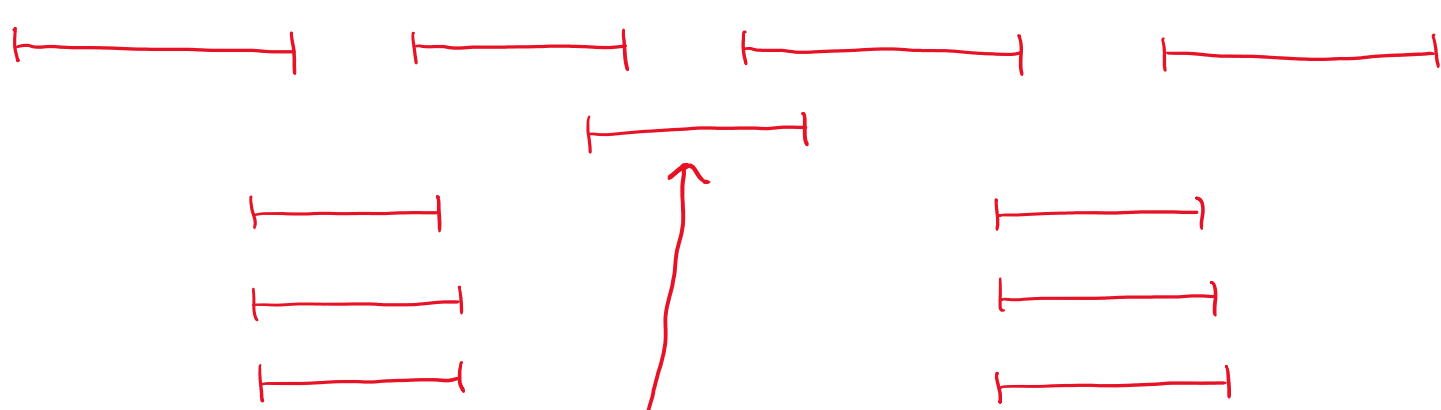
- ① FCFS (earliest start time)



- ② Shortest time interval



- ③ Fewer incompatibilities



Algo chooses
Optimal: four in the top row